Práctica 2
Manejo Avanzado de *Flex*

Jordi Bernad, Elvira Mayordomo, José Manuel Colom, Carlos Bobed, Gregorio de Miguel, Carlos Hoyos

Tareas

1. Estudie la sección sobre las *condiciones de arranque* en el capítulo 9 del manual de *Flex* disponible en moodle y en el libro *flex & bison* (páginas 28 a 31)
2. Lea la introducción de esta práctica y realiza los ejercicios propuestos.
3. Elabore la memoria de la práctica y entrégela junto con los ficheros fuente según el Procedimiento de Entrega de Prácticas explicado en la Introducción a las Prácticas de la Asignatura. La fecha tope de entrega será hasta el día anterior al comienzo de la Práctica 3.

Nota: El incumplimiento de las normas de entrega se reflejará en la calificación de la práctica.

Se recuerda especialmente lo siguiente:

- Verifique que todos tus ficheros fuente (*.l*) contienen como comentario en sus primeras líneas los NIAs y nombres de los autores. Todos los programas deberán estar debidamente documentados.
- Verifique que los ficheros que se presentan compilan y ejecutan correctamente en lab000.
- Cree un fichero comprimido .zip llamado

  `niaPr2.zip`

  donde *nia* es el identificador personal. Este fichero comprimido **debe contener exclusivamente** el fichero con la memoria en formato *PDF*, los ficheros fuente con tu código (*.l* de *Flex*), y los de prueba (*.txt* de texto). No usar subdirectorios.
- Utilice el enlace a una tarea de moodle disponible en la sección "Prácticas de Laboratorio" para entregar el fichero `niaPr2.zip`
- Recuerde incluir en la memoria una sección declarando el uso de herramientas de Inteligencia Artificial.

Introducción

Los objetivos de esta práctica son:

- Aprender a desarrollar analizadores léxicos en *Flex* más sofisticados, profundizando en el manejo de lo que se conoce como *condiciones de arranque*. Las *condiciones de arranque* no son imprescindibles, ya que siempre se pueden emular utilizando código *C* en las acciones de los patrones. No obstante, su uso facilita mucho el desarrollo de los programas en *Flex,* ya que ayudan a estructurar conjuntos de patrones/acciones en función de un contexto determinado.
- Aplicar los conocimientos teóricos de Autómatas Finitos para desarrollar programas.

Ejercicio 1

Implemente un analizador lexicográfico en *Flex* (fichero *ej1.l*) que tome como entrada un fichero fuente de *Flex* **sintácticamente correcto** (tanto el código de *Flex* como el código de C insertado) con las siguientes restricciones:

- los patrones de las reglas no contendrán espacios en blanco o tabuladores, ni estarán divididos en varias líneas;
- las acciones de las reglas estarán siempre entre `{` y `}`;
- en la sección de reglas, los caracteres `{` y `}` solo aparecen: como parte del patrón de una regla; para delimitar el bloque de acciones de una regla; o dentro de las acciones para delimitar bloques de código (bloques if, for, etc).

y devuelva la siguiente información (el total de reglas y una aproximación al número de líneas escritas en C):

1. De la primera sección (sección declaraciones): total de líneas (total de caracteres `\n`) dentro de los bloques de C: bloques marcados entre `%{` y `%}` al inicio de una línea. Nótese que pueden aparecer varios de estos bloques en un fichero de *Flex* sintácticamente correcto.
2. De la segunda sección (sección reglas):
   - Número total de líneas dentro de todas las acciones de las reglas: Consideraremos que el total de líneas en un bloque de acciones es el número de saltos de línea que hay entre el carácter `{` que abre el bloque y el carácter `}` que lo cierra más uno.
   - Número total de reglas.
3. De la tercera sección (sección código): total de líneas. La línea que marca el inicio de esta sección se contabilizará como línea de esta sección.

Para simplificar el problema, suponga también que todas las constantes de tipo cadena de caracteres de C están delimitadas por doble comillas (carácter ") y que dentro de estas constantes no aparece el carácter ", es decir, no aparecerán cadenas del estilo

```
printf(" muestro \" por pantalla ")
```

En la resolución del problema tenga en cuenta que dentro de las constantes de cadena y

de los dos tipos de comentarios de C, comentarios de línea `//...` y multilínea `/*....*/`, puede aparecer cualquier carácter, como llaves de apertura y cierre, porcentajes, etc.

La salida del programa constará exactamente de 4 líneas de texto con el formato:

```
C1:<total líneas en los bloques de C>
C2:<total líneas en acciones>
C3:<total líneas en sección código>
R:<total reglas>
```

Ejemplo:

Entrada

```
  //mi nombre
%{
#include<math.h>  //comentario%}%}%}

int a = 0;
%}

DIGITO [0-9]

%{
int b = 0;
%}

%%
{DIGITO}+     {printf("hola}}}");
a++;}

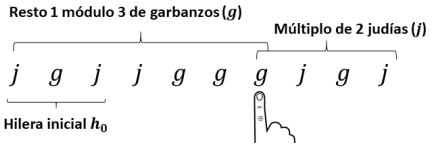
ggg                  {}
"aabb{"[A-Z]+   { a++;
 if (a > b) { /*comentario  varias lineas
    }} fin comentario*/  b++;}
}
%%
int main() {
    yylex();
}
```

Salida

```
C1:6
C2:7
C3:4
R:3
```

Ejercicio 2

Consideremos el siguiente juego por turnos para dos jugadores. Cada jugador dispone de 20 semillas: 10 garbanzos (g) y 10 judías (j). El juego comienza con una hilera inicial de semillas h_0. En cada turno: primero, el jugador añade una de sus semillas a la derecha de la hilera; segundo, si el jugador es capaz de señalar una semilla tal que entre las semillas desde la marcada (**incluida**) hasta al inicio de la hilera hay un número de garbanzos con resto 1 módulo 3, y desde la marcada (**incluida**) hasta al final de la hilera hay un número par de judías, puede elegir entre repetir turno o dejarle el turno al adversario. Por ejemplo, en la figura de abajo, el jugador habría colocado una judía (última semilla a la derecha); señala una semilla (el garbanzo marcado con el dedo), y como hay un múltiplo de 3 más uno de garbanzos (cuatro garbanzos) desde la semilla señalada hasta el inicio, y un número par de judías (dos judías) desde la señalada al final, el jugador puede repetir turno (o no) poniendo otra semilla. Gana el jugador que termina antes de colocar todas sus semillas en la hilera.



El objetivo del ejercicio es identificar mediante expresiones regulares las hileras h_0 para las que el jugador que comienza la partida (jugador 1) tiene una estrategia ganadora, y las h_0 para las que es el otro jugador (jugador 2) el que posee una estrategia ganadora.

Implemente un analizador lexicográfico utilizando Flex (fichero *ej2.l*) que dado un fichero de texto que contiene en cada línea una secuencia de g y j, añada al inicio de cada línea `1-` o `2-`, si la línea representa un h_0 para el que existe estrategia ganadora para el jugador 1 y jugador 2, respectivamente.

Se puede suponer que en el texto de entrada solo aparecen los caracteres g y j junto con los saltos de línea.

Ejemplo:

Entrada	**Salida**
`jjgj`	`1-jjgj`
	`1-`
`ggjj`	`2-ggjj`
`jjgg`	`2-jjgg`
`jjggj`	`1-jjggj`
`jgj`	`1-jgj`

Observe que la segunda línea del fichero de entrada del ejemplo está vacía: situación en la que se comienza el juego sin ninguna semilla ($h_0 = \epsilon$).

Incluya en la memoria las explicaciones pertinentes de cómo se ha deducido la solución.

Ayuda: Sobre el alfabeto $\Sigma = \{g, j\}$, considere el lenguaje de las cadenas para las que existe un símbolo $x \in \Sigma$ tal que desde el inicio de la cadena hasta x contiene un múltiplo de 3 más 1 de g, y desde x hasta el final de la cadena contiene un número par de j, esto es,

$$L = \{w \in \Sigma^* \mid w = u_1 x u_2,\ |u_1 x|_g = 3n+1,\ |x u_2|_j = 2m \ \ u_1, u_2 \in \Sigma^*,\ x \in \Sigma,\ n, m \in \mathbb{N}\}$$

Ayudándose de JFlap, diseñe el AFD mínimo que reconoce el lenguaje L (es más fácil diseñar primero un AFnD y pasar a un AFD). Utilice este autómata (sus estados) para identificar los h_0 para los que hay una estrategia ganadora para cada uno de los jugadores. Modifique el autómata para hallar las expresiones regulares (de nuevo, utilice JFlap) que identifican hileras h_0 desde las que existe una estrategia ganadora para el jugador 1 y 2. Recuerde que las expresiones regulares que proporciona JFlap se adaptan a la sintaxis explicada en las clases de teoría, pero no a la sintaxis de Flex.